Genetic Programming for Evolutionary Feature Construction in Symbolic Regression

Hengzhe Zhang

Victoria University of Wellington

26 February 2026

Outline

1. Introduction
2. Chapter 3: Semantic-Aware Bloat Control
3. Chapter 4: Vicinal Risk Decomposition
4. Chapter 5: Neural-Guided Search
5. Chapter 6: Ensemble Learning
6. Conclusions

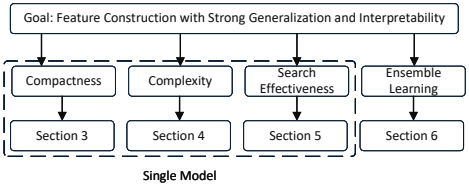
The Problem: Feature Construction in Symbolic Regression

Symbolic Regression (SR):

- Discover mathematical expressions from data
- Learn $f : X \to Y$ in symbolic form
- Both *structure* and *parameters* are searched

Feature Construction:

- Construct $\Phi(X) = \{\phi_1(X), \ldots, \phi_m(X)\}$ from raw features
- Combined via linear regression: $\hat{Y} = \sum_j \beta_j \phi_j(X)$



Motivation: Why feature construction-style SR?

Why FEAT[1], ITEA[2], FFX[3] use feature construction:

- **Efficient, closed-form solution** — coefficients β_j fit by linear/ridge regression, no iterative optimization
- **Analyze important features** — rank and interpret features $\phi_j(X)$ rather than one monolithic expression
- **Compositional search** — good performance via interacting features; no need to discover a single powerful expression

Bloat →	SHM
Overfitting →	VJM-GP
Slow Search →	RAG-SR
Fragility →	SR-Forest

Key Challenges: Vast search space, bloat, overfitting, inefficient search, robustness

[1] La Cava & Moore (2020). Learning feature spaces for regression with GP. *GPEM* 21(3):433–467.
[2] de França & Aldeia (2021). Interaction–transformation evolutionary algorithm for SR. *Evol. Comput.* 29(3):367–390.
[3] McConaghy (2011). FFX: Fast, scalable, deterministic symbolic regression. *GPTP IX*, Springer.

Research Goals and Objectives

*How can we enhance GP-based feature construction to achieve both good **parsimony** and **generalization**?*

1. **Parsimony:** Semantic-aware operators to reduce tree size without compromising accuracy
2. **Generalization:** Principled regularization by decomposing complexity beyond tree size
3. **Search Effectiveness:** Neural networks to accelerate GP search with accumulated knowledge
4. **Robustness:** Leverage GP's population to build heterogeneous ensembles

Thesis Approach

Four complementary innovations within a unified multi-tree GP feature construction framework. They can be combined in one framework (e.g. RAG-SR+SHM). Full integration of all four is possible.

Chapter 3: Semantic-Aware Bloat Control

Semantic Hoist Mutation (SHM)

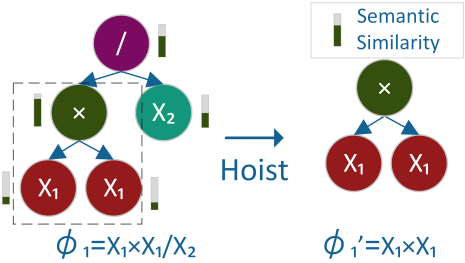
Key Insight: *Not all subtrees are equal!*

Traditional Approach:

- Random hoist mutation or size-based penalties
- No consideration of subtree *semantics*

Our Approach — SHM:

1. Measure semantic similarity of each subtree to the target:
$$\theta = \cos\big(\phi_{\text{sub}}(X),\ Y\big)$$
2. **Hoist the most informative subtree** (highest θ)
3. **Hash-based diversity:** $O(1)$ duplicate detection to prevent redundant features



Effect: Trees reduced from ∼20 nodes to ∼4.5 nodes

SHM: Theoretical Foundation and Results

Generalization Bound:

$$L_{exp}(\phi) \leq (1-\theta^2)\left(1-\sqrt{p-p\ln p+\frac{\ln n}{2n}}\right)_+^{-1}$$

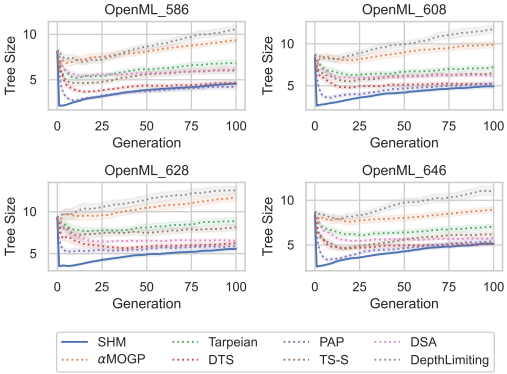
Key Insight:

- Higher $\theta \Rightarrow$ tighter bound
- Semantic similarity **directly impacts** generalization
- Justifies semantic-based subtree pruning

Results:

- **Error:** test R^2 maintained
- **Size:** significant reduction (e.g. $\sim$20 to $\sim$4.5 nodes)
- Outperforms other bloat control methods

Published in IEEE TEVC 2024



Average tree sizes across generations

Chapter 4: Vicinal Risk Decomposition

Beyond Model Size: Functional Complexity

Traditional View:

- Overfitting $\leftrightarrow$ Large models
- Control via parsimony pressure

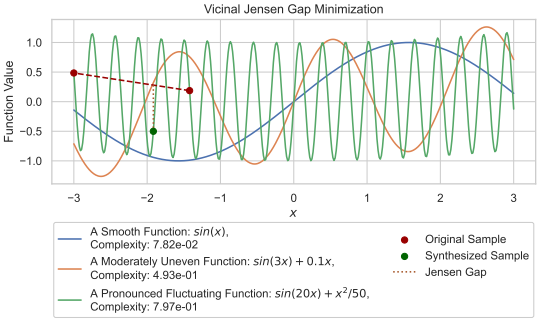
Our Observation:

- $x_1 \times x_2$ vs $\sin(\sin(x))$
- Same tree size, **different complexity!**
- Need *functional* complexity measures

Key Innovation:

Decompose vicinal risk into:

$$\underbrace{O_1}_{\text{Empirical Loss}} + \underbrace{O_2}_{\text{Functional Complexity}}$$



Vicinal Risk Decomposition: Theory and Results

Decomposition:

$$\mathcal{V}(f) = \underbrace{\frac{1}{n}\sum_i (f(x_i) - y_i)^2}_{O_1\text{: Emp. Loss}} + \tau \cdot \underbrace{\text{Jensen Gap}}_{O_2\text{: Reg.}}$$

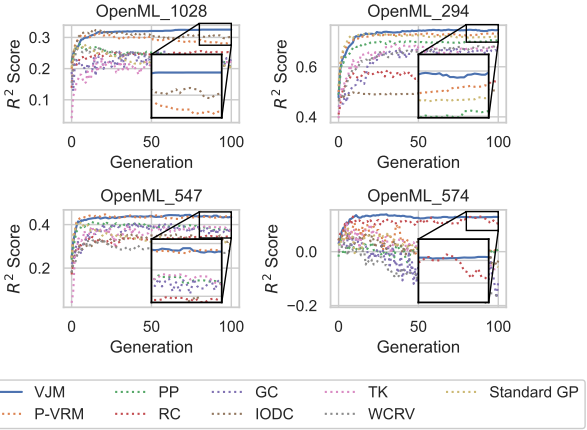
Two Instantiations:

- **Mixup:** O_2 = Vicinal Jensen Gap
  Promotes local linearity
- **Perturbation:** O_2 = Finite Difference
  Controls sensitivity

Adaptive Regularization:

- Noise estimation (Extra Trees CV)
- Manifold intrusion detection

Published in IEEE TEVC 2025



Test R^2 scores across generations

Chapter 5: Neural-Guided Search

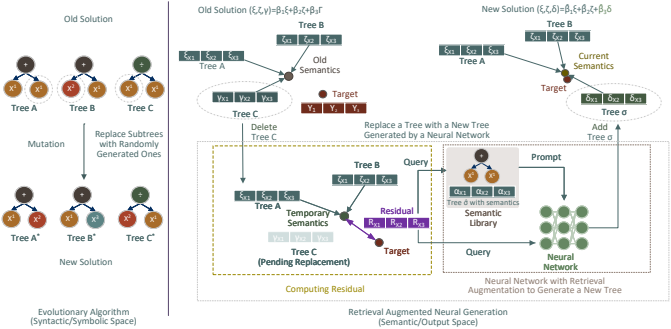
RAG-SR: Retrieval-Augmented Generation for SR

GP: No memory across generations
Neural SR: Hallucination problem

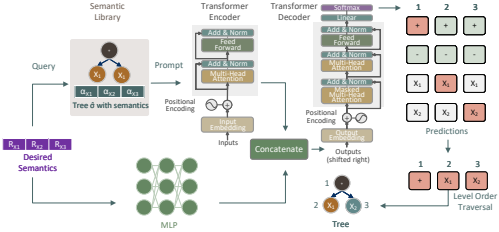
Solution — RAG-SR:

1. Build **symbolic library** during evolution
2. **Retrieve** similar expression
3. **Generate** improved expression via neural decoder

Retrieval **grounds** generation, reducing hallucination



RAG-SR: Architecture and Results



MLP encodes semantics. Transformer Encoder encodes retrieved tree. Decoder generates new expression.

Key Innovations:

- **Semantic Descent:** iteratively replace trees to minimize residual
- **Scale invariance:** sign-insensitive training
- **Online learning:** model trained during evolution

Results:

- Outperforms pre-trained neural SR and state-of-the-art GP methods
- Closer to ground-truth expressions

Published in ICLR 2025

Chapter 6: Ensemble Learning

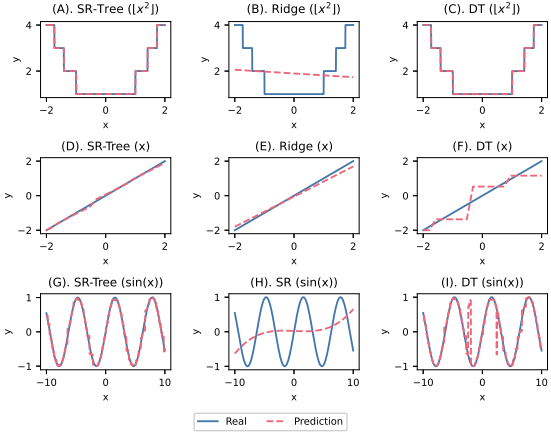
SR-Forest: A Heterogeneous Ensemble

Motivation:

- GP is inherently **population-based**
- Different models capture different patterns
- Bias–Variance trade-off:
  $$E_F = \underbrace{\mathbb{E}[(f - Y)^2]}_{\text{avg. loss}} - \underbrace{\mathbb{E}[(f - \bar{F})^2]}_{\text{ambiguity}}$$

SR-Tree = GP $\oplus$ Linear $\oplus$ DT:

- GP features: smooth, compact
- Linear regression: continuous relationships
- Decision tree: piecewise/discontinuous patterns
- **Complementary strengths**



Different functions need different model types — SR-Tree handles both continuous and piecewise patterns.

SR-Forest: Key Innovations and Results

Three Innovations:

1. **Guided Mutation:** probabilistic feature selection weighted by importance
2. **Multi-Fidelity:** Ridge-only for 80% of generations, full SR-Tree for 20% (1.8× speedup)
3. **Ensemble Selection:** error reduction-based, bagged for robustness

Results:

- Outperforms 25 existing SR and ensemble methods
- Effective on both synthetic and real-world data

Top-5: Synthetic & Real-World (median R^2, rank)

Algorithm	Synthetic (Rank)	Real-World (Rank)
SR-Forest	**0.9674 (1)**	**0.7168 (1)**
Operon	0.9516 (2)	0.6323 (17)
SBP-GP	0.9257 (3)	0.6698 (8)
GP-GOMEA$_{\mathrm{SRF}}$	0.9134 (4)	0.6721 (7)
FEAT	0.9107 (5)	0.6597 (11)

Published in IEEE TEVC 2024

Conclusions

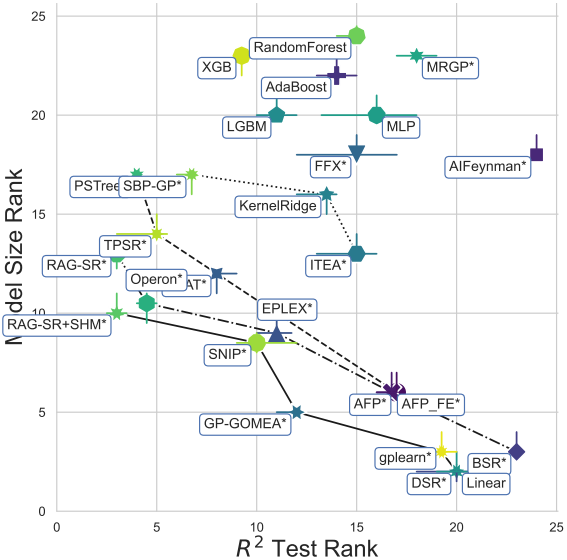
Integration: RAG-SR + SHM Case Study

Case Study: RAG-SR + SHM

- Merged Ch. 5 (RAG-SR) + Ch. 3 (SHM) into **RAG-SR+SHM**
- Achieves **smaller models**, competitive accuracy
- Confirms SHM bloat control within neural-guided GP

Potential Integrations:

- VJM: integrate for small/noisy data
- SR-Forest: integrate when parsimony is not the priority



Summary of Contributions

Challenge	Solution	Key Innovation	Venue
Bloat	SHM	Semantics guide simplification	TEVC 2024
Overfitting	VJM-GP	Decompose complexity	TEVC 2025
Search Efficiency	RAG-SR	Neural memory + retrieval	ICLR 2025
Robustness	SR-Forest	Heterogeneous ensemble	TEVC 2024

Core Achievement

Advanced GP-based feature construction through four complementary innovations addressing **parsimony**, **generalization**, **search effectiveness**, and **robustness**—each rigorously validated and designed for composability.

Conclusions and Future Work

This thesis demonstrates that GP-based feature construction can achieve both good parsimony and generalization through:

1. Semantic-aware operations that reduce complexity without sacrificing accuracy
2. Theoretical decompositions that enable flexible overfitting control
3. Neural integration that accelerates search
4. Ensemble methods that leverage population-based strengths to improve robustness

Future Directions:

- RAG-SR+SHM demonstrates cross-chapter integration, extending to include VJM
- LLM-driven algorithm design for GP operators

Questions?

Thank you for your attention!

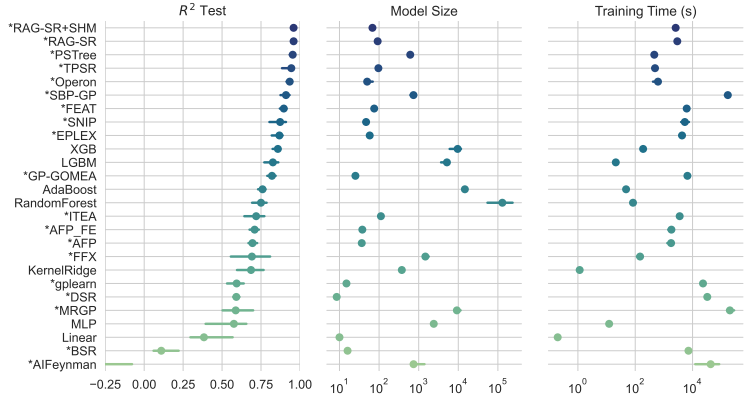
All methods are open-sourced and available at:
`github.com/hengzhe-zhang`

Appendix

Backup Slides for Q&A

Response: Integration Results — RAG-SR+SHM

Q (Grant, Major): "a lack of integration of these concerns into a single framework."
Q (Bach, General, no. 4): "it is not always clear why techniques proposed in earlier chapters are not utilised or extended in later chapters."



Response: Interpretability and Parsimony

Q (Grant, Major): "A fundamental criticism of the work is in its definition of 'interpretable'. It seems that this concept is weakly defined."
Q (Ting, Ch. 3.5): "does compactness necessarily imply greater interpretability?"

Position:

- We agree: the results are best characterized as **parsimony** (i.e. compact size), a **necessary but not sufficient** condition for interpretability.
- Formal interpretability metrics remain an open research problem.

Response: Interpretability in SR-Forest

Q (Ting, Ch. 6): "How the tension between ensemble performance and interpretability should be understood in SR-Forest?"
Q (Grant, Ch. 6): SR-Forest chapter — increase in difficulty interpreting SR-Tree over ridge regression

A:

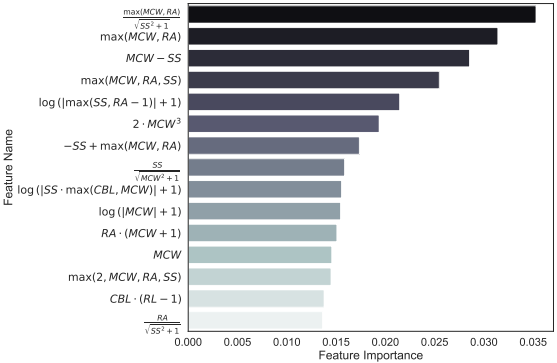
1. There is a trade-off between **interpretability** and **effectiveness**. In the ensemble setting, interpretability is addressed primarily through the **importance of constructed features**: **feature importance** indicates which GP features contribute more to the model.
2. Although a single SR-Tree is more complex than ridge regression alone, SR-Forest remains an ensemble. From a **feature-importance** analysis perspective there is no added interpretability cost—one still inspects which **features and weights** matter.

A case study of feature importance on SR-Forest

Task: Predict XGBoost performance from hyperparameters

SR-Forest discovered:

- Min child weight (MCW) crucial
- L1 regularization (RA) important
- Key: Max(MCW, RA)
- Aligns with theory[4]



Feature importance shows which **constructed features** contribute.

[4] Cortes et al., "Regularized Gradient Boosting," NeurIPS, 2019.

Response: Wrapper-Based vs Embedded GP

Q (Bach, General, no. 1): "it would be important to further justify the necessity of adopting a wrapper-based framework, which is computationally more expensive."
Q (Bach, General, no. 3): "the rationale for choosing Linear Regression as the wrapped model"

Advantages: LR yields a **closed-form solution**. Coefficient optimization within complex GP expressions is problematic due to **gradient vanishing and explosion**.
Rationale for LR:

- LR is one of the most interpretable models
- Closed-form solution is very fast

Response: Wrapper-Based vs Embedded GP (cont.)

Q (Bach, General, no. 2): "how does this compare to embedded GP approaches that do not require pre-specifying the number of constructed features?"

A:

1. Embedded GP is not adopted because coefficient optimization within complex GP expressions leads to **gradient vanishing and explosion**. The wrapper with LR avoids this through a **closed-form solution**.
2. Tree addition and deletion operators allow a **variable number of trees** in wrapper-GP but introduce additional **hyper-parameters**.

Response: Choice of PMLB Benchmarks

Q (Ting, Ch. 3.3.1): "Why was PMLB chosen as the primary benchmark repository?"
Q (Grant, Ch. 4): " 'inherent complexity' needs definition — what is complex about them?"

Why PMLB: Using PMLB allows us to reuse results in SRBench.

Why real-world over synthetic:

- Matsubara et al.[5]: synthetic benchmark sampling is **oversimplified** and does not reflect real-world settings
- Haut et al.[6]: **37 of 100** synthetic equations can be recovered with only three randomly sampled points, indicating insufficient complexity

On "inherent complexity": The term was imprecise. Real-world data exhibit **noise**, **uneven sampling**, and **unknown functional forms** that synthetic benchmarks often lack.

[5] Matsubara et al., "Rethinking Symbolic Regression Datasets and Benchmarks for Scientific Discovery," *DMLR*, 2024.
[6] Haut et al., "Active Learning Informs Symbolic Regression Model Development in GP," *GECCO Companion*, 2023.

Response: Role of the MLP in RAG-SR

Q (Grant, Ch. 5): "a more thorough exploration of what the MLP component actually does"
Q (Bach, Ch. 5, no. 1): "The role of the MLP should be more clearly explained"

Role of the MLP: The MLP encodes the **desired semantics**. Without it, they cannot be supplied to the **decoder**.

Response: Hashing Method in SHM

Q (Grant, Ch. 3): "How is the hash computed? ...is the hash on the tree structure or the semantics?"
Q (Bach, Ch. 3, no. 3): "The hashing method used to distinguish trees should be described in more detail"

A:

1. The hash is on the **tree structure** (syntax), not on semantics.
2. Full **semantic comparison** is computationally too expensive. **Locality-sensitive hashing** can compress semantics but was found to add complexity without improvement in experiments.

Background Slides

Regression: The Fundamental Task

Definition:

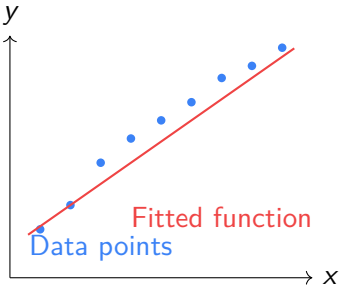
- Predict continuous output Y from input X
- Learn function $f: X \to Y$
- Minimize prediction error

Common Loss Function:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2$$

Evaluation Metrics:

- Mean Squared Error (MSE)
- R^2 score: $1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$



Key Challenge: Finding the right functional form

Linear Regression

Model Form:

$$f(x) = \beta_0 + \sum_{j=1}^{m} \beta_j x_j$$

Optimization:

$$\min_{\beta} \| Y - X\beta \|^2$$

Closed-form Solution:

$$\beta = (X^T X)^{-1} X^T Y$$

With Regularization (Ridge):

$$\beta = (X^T X + \lambda I)^{-1} X^T Y$$

Advantages:

- Simple and interpretable
- Fast computation
- Well-understood theory

Limitations:

- Assumes linear relationship
- Cannot capture interactions
- Poor for non-linear patterns

Bloat in Genetic Programming

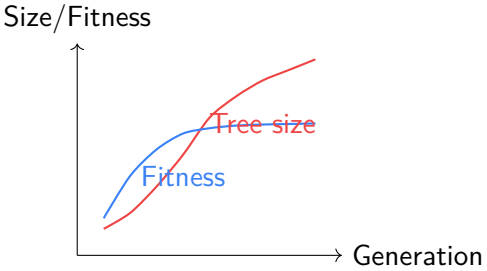
Definition: Growth of program size without corresponding improvement in fitness

Hypotheses for Bloat:

- **Hitchhiking:** Neutral code attaches to fit individuals
- **Defense:** Large trees resist crossover damage
- **Removal bias:** Removing small subtrees is less harmful
- **Search space:** Larger programs more common

Consequences:

- Reduced interpretability
- Increased evaluation time
- Memory consumption
- Search inefficiency



Common Control Methods:

- Depth/size limits
- Multi-objective (size vs fitness)
- Selection operators
- Simplification operators

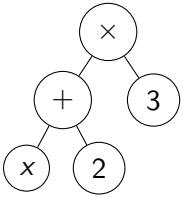
Semantics in Genetic Programming

Syntax vs Semantics:

- **Syntax:** Structure of the program (tree)
- **Semantics:** Behavior/output of the program

Formal Definition:
For a GP tree ϕ and dataset X:

$$\text{Semantics: } s(\phi) = [\phi(x_1), \phi(x_2), ..., \phi(x_n)]$$



Syntax: Tree

Example:

- Tree 1: $(x + 2) \times 3$
- Tree 2: $3x + 6$
- Different syntax, same semantics!

For dataset: $X = \{1, 2, 3\}$

$$s(\phi) = [9, 12, 15]$$

Overfitting in Machine Learning and GP

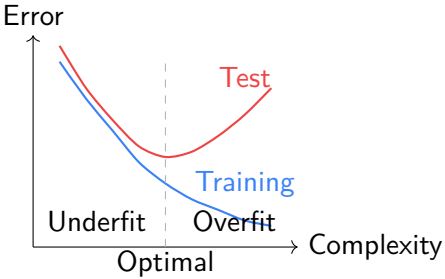
Definition: Model performs well on training data but poorly on unseen data

Causes in GP:

- Complex expressions
- Noise fitting
- Limited training data
- Excessive generations

Detection:

- Training vs test error gap
- Cross-validation performance
- Complexity growth



Control Strategies:

- ✓ Regularization (L1, L2, complexity)
- ✓ Cross-validation
- ✓ Data augmentation
- Ensemble methods

Model Complexity: Size vs Function

Tree Size Complexity:

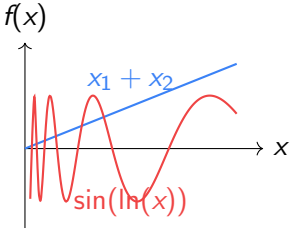
- Number of nodes
- Tree depth
- Number of operations

Functional Complexity:

- Smoothness/roughness
- Number of oscillations
- Sensitivity to input changes
- Curvature

Key Insight:
Size $\neq$ Functional Complexity

Example: Same Size (3 nodes)



Complexity Measures:

- VC dimension
- Rademacher complexity
- Jensen gap
- Lipschitz constant

Mixup: Data Augmentation Technique

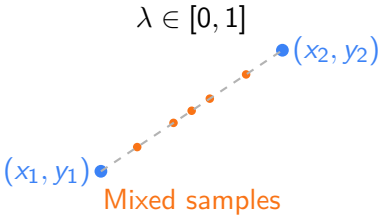
Core Idea: Train on convex combinations of examples

Formula:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j \quad (1)$$

$$\tilde{y} = \lambda y_i + (1-\lambda)y_j \quad (2)$$

$$\lambda \sim \text{Beta}(\alpha, \alpha) \quad (3)$$



Beta Distribution:

- $\alpha = 1$: Uniform
- $\alpha < 1$: Favor extremes
- $\alpha > 1$: Favor center

Common: $\alpha \in [0.2, 2]$

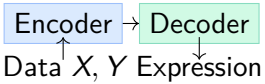
Benefits:

- Regularization effect
- Smoother regression function
- Better generalization
- Simple to implement

Neural Networks for Symbolic Regression

Neural SR Approaches:

1. **Pre-trained Models:**
   - Train on synthetic datasets
   - Fast inference
   - Limited to training distribution
2. **Reinforcement Learning:**
   - Learn policy for tree generation
   - Flexible function set
   - Sample inefficient
3. **Seq2Seq Models:**



Challenge: Hallucination

- Syntactically correct
- Semantically wrong

Retrieval-Augmented Generation (RAG)

Traditional Generation:

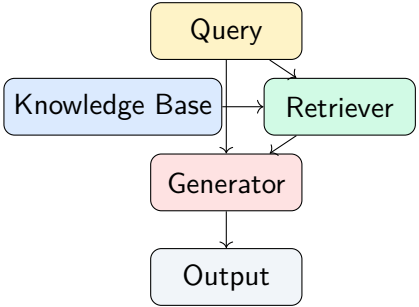
- Code generates from parameters
- Risk of hallucination
- Limited by training data

RAG Framework:

1. **Retrieve:** Find relevant examples
2. **Augment:** Add to context
3. **Generate:** Produce output

Components:

- Knowledge base/library
- Retrieval mechanism (k-NN, dense)
- Generation model



Advantages:

- Reduces hallucination
- Updatable knowledge
- Better grounding
- Interpretable retrieval

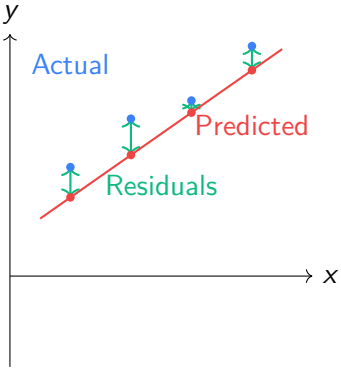
Residuals in Regression

Definition:

$$\text{Residual: } r_i = y_i - \hat{y}_i$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value



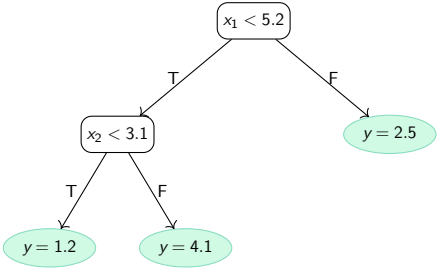
Decision Trees

Structure:

- Binary tree of decisions
- Internal nodes: split conditions
- Leaf nodes: predictions
- Axis-parallel splits

Algorithm (CART):

1. Select best split (MSE/Gini)
2. Partition data
3. Recurse on subsets
4. Stop at criterion



Properties:

- Piecewise constant
- Non-parametric
- Handles non-linearity

Advantages: No scaling needed

Limitations: Overfitting, axis-parallel only

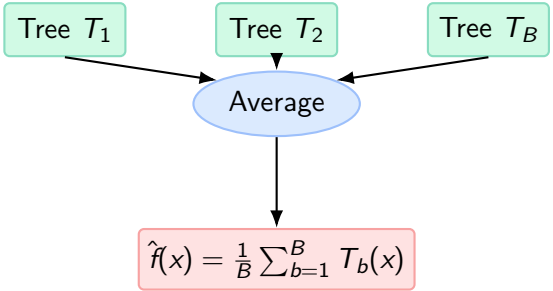
Random Forests: Bias–Variance Tradeoff

Bias–Variance Decomposition:

$$\mathbb{E}\left[(Y-\hat{f}(x))^2\right] = \underbrace{\left(\mathrm{Bias}[\hat{f}(x)]\right)^2}_{\text{systematic error}} + \underbrace{\mathrm{Var}[\hat{f}(x)]}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

Random Forest intuition:

- Each tree: low bias, high variance
- Averaging many trees $\Rightarrow$ variance drops
- Bias remains low (deep trees, no pruning)
- Net effect: lower test error



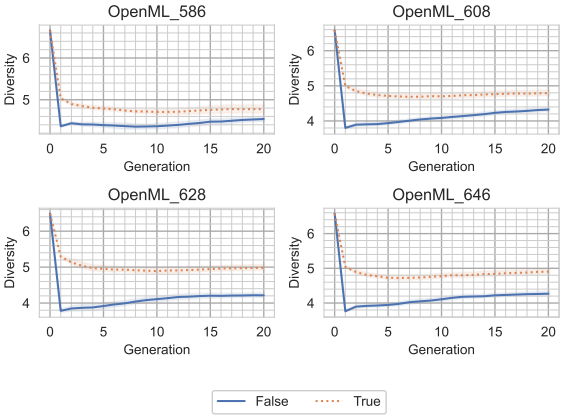
Detailed Methodology Slides

Maintaining Diversity: Hash-based Checking

Problem: Hoisting best subtree repeatedly → Duplicate features

Solution: Hash-based duplicate prevention

- $O(1)$ checking complexity
- Maintains feature diversity
- Critical for multi-tree GP

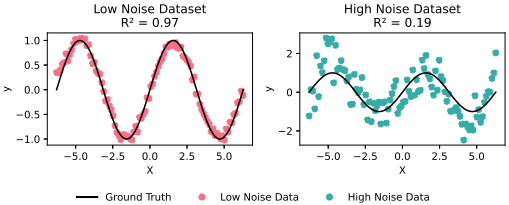


Result: Higher phenotype entropy throughout evolution

Smart Regularization: Noise Estimation & Manifold Detection

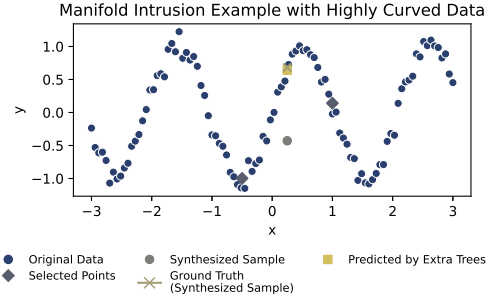
Noise Estimation:

- Use Extra Trees CV score
- Adapt weight τ accordingly
- High noise $\rightarrow$ More regularization

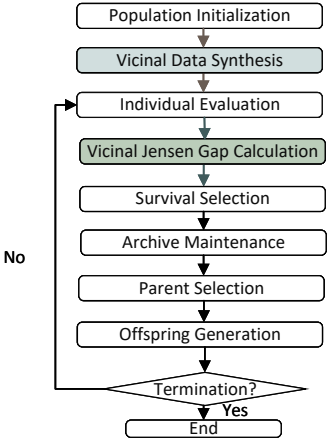


Manifold Intrusion Detection:

- Synthesized data may be unrealistic
- Check with reference model
- Discard intrusion points



VJM-GP: The Complete Framework



Key Components:

- Multi-objective optimization: O_1 (accuracy) vs O_2 (complexity)
- Lexicase selection for semantic diversity
- Environmental selection via non-dominated sorting
- Adaptive regularization based on dataset characteristics

Semantic Descent: Iterative Refinement

Core Idea: Replace trees to minimize residual

Algorithm:

1. Remove tree ϕ_i contribution
2. Compute residual $R = Y - \Phi^{temp}(X)$
3. Generate replacement:
   - Neural: $\mathcal{N}(R, \mathcal{L})$
   - Retrieval: Best from library
4. Update if improved

Key Innovation:

- **Replace** not add (maintain size)
- Online learning during evolution
- Leverage accumulated knowledge

Handling Scale Invariance

Challenge: Linear regression adjusts magnitude automatically

Data Augmentation:

- Include $(\phi, \phi(X))$ and $(\phi, -\phi(X))$
- Sign-insensitive learning

Double Query:

- Query with both R and $-R$
- Select highest probability

Masked Contrastive Loss:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{L}_{InfoNCE}$$

Aligns semantic embeddings with symbolic representations

Achievement

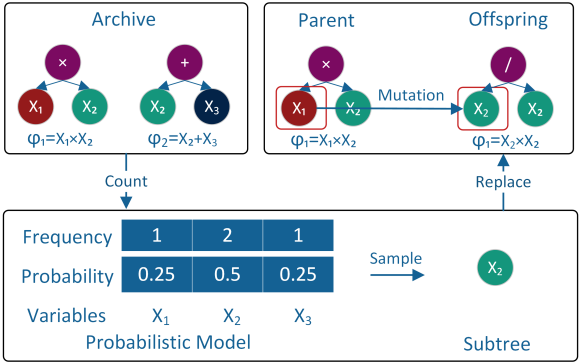
Online-trained neural SR+GP outperforms pre-trained models

Innovation: Guided Mutation for High Dimensions

Problem: Random mutation inefficient with many features

Solution: Probabilistic guidance

- Track feature importance
- Weight by frequency × importance
- Dynamically updated

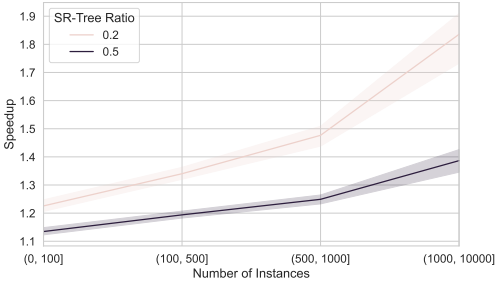


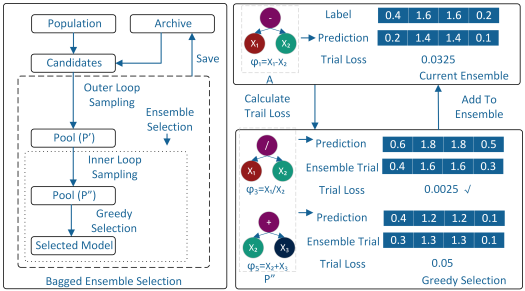
Impact: Significantly better performance on high-dimensional data

Multi-fidelity Evaluation & Ensemble Selection

Multi-fidelity Evaluation:

- First 80% generations: Ridge only
- Last 20%: Full SR-Tree
- 1.8× speedup achieved



Ensemble Selection:

- Error reduction-based
- Bagged selection (robustness)
- Semantic diversity preserved